

AI/ML BOOTCAMP

(Day -2)

Ist concept: *Qualities to become an AI engineer*

Let us understand with some scenario:

- ✓ *Scenario 1: Twitter chat bot (Tay)*
- ✓ *Scenario 2: A robot*

Scenario 1:

In 2016, Microsoft launched an AI chatbot called Tay on Twitter.

It was designed to learn from user conversations and become more intelligent over time. Within 16 hours of going live, it had to be shut down. Tay had begun generating racist, abusive, and politically extreme content — not because it was hacked, not because the engineers were incompetent — but because it did exactly what it was programmed to do. It learned from users. Users gamed it. It learned what it was taught

Scenario 2:

Consider this instruction given to a robot in a warehouse: "Pick up all boxes weighing more than 10 kg and move them to Zone B." The robot picks up a box — that happens to contain fragile glassware — weighing 12 kg — and drops it in Zone B at full speed. The glassware is destroyed. The robot did nothing wrong. The instruction was followed precisely. The engineer assumed "boxes" meant "safe to handle at normal speed." The machine made no such assumption — it has none.

What went wrong in both case?

- *In case of chatbot, they did not take care of the assumptions. They did not assume that chatbot can learn those type of things too which lead to failure.*
- *In case of robot, they did not give proper prompt to the robot, which in turn lead to blunder.*

Conclusion:

Relationship between Code/Data/Assumptions must be considered.

2nd concept: *Difference between code, program, software, software product?*

What is programming?

- *Programming is the process of designing and writing a set of instructions, expressed in a formal language, that a computing machine can interpret and execute to perform a defined task.*
- *Programming is the discipline of converting human intent into machine-executable logic, with full responsibility for every assumption made during that translation.*
- *In the context of Artificial Intelligence, programming is not just about instructing machines what to do — it is about designing systems that can infer, generalize, and decide based on patterns in data, within the boundaries of logic defined by the programmer.*

This distinction matters. Classical programming: if condition $\rightarrow$ action . Machine learning: given data $\rightarrow$ learn condition $\rightarrow$ infer action . The programmer shifts from writing rules to designing the environment in which rules are discovered.

This has a profound implication for AI/ML: When your ML model makes a "wrong" decision — it is not the model being wrong. It is the model correctly executing the logic it was trained on, with data it was trained on, within the architecture it was designed with. The failure is almost always upstream — in data quality, problem framing, assumption validation, or architectural choice. This is why debugging mindset is not optional in AI/ML. It is the core skill.

Hidden Insights:

Industry surveys consistently show that experienced engineers spend roughly 60–80% of their time reading existing code, debugging, and understanding failures — not writing new code. In AI/ML specifically, a significant portion of a practitioner's time is spent on: understanding why a model isn't converging, diagnosing data leakage, interpreting unexpected metrics, and tracing errors through pre-processing pipelines. Writing code is the easy part. Understanding what you wrote — and why it behaves the way it does — is the skill.

What is Software? Code? Program?

Software is not a synonym for programming. This distinction is worth stating explicitly.

Code: *Individual instructions written in a programming language.*

Program: *A cohesive set of instructions that performs a defined task.*

Software: *A system of programs, data, configuration, and documentation that delivers a capability to users at scale.*

Software Product: *Software designed, tested, deployed, and maintained for use by others outside the development team.*

4th concept: Why Python ?

Every programming language is created with a purpose, and Python is no exception. Understanding the story behind its creation explains why it looks and behaves the way it does today.

The Origin Story

Python was created by Guido van Rossum and released in 1991. He started working on it during the Christmas holidays of 1989 at Centrum Wiskunde & Informatica (CWI) in the Netherlands, as a hobby project to keep himself occupied during the break. At the time, he was working on the ABC programming language and wanted to build something that fixed ABC's limitations — particularly around extensibility and exception handling.

The name "Python" has nothing to do with the snake. Guido van Rossum was a fan of the British comedy show "Monty Python's Flying Circus," and he wanted a name that was short, unique, and slightly mysterious — so he picked Python. This is reflected even today in Python's culture, where the official documentation and community often use Monty Python references as examples (e.g., variable names like spam and eggs).

Why Python Was Designed the Way It Was

Guido's core motivation was readability and simplicity. He believed that code is read far more often than it is written, so the language should prioritize human understanding over machine efficiency. This philosophy is captured in "The Zen of Python" (PEP 20), a set of guiding principles for the language, which includes lines such as "Readability counts" and "Simple is better than complex."

Key Characteristics of Python

Interpreted Language: Code is executed line by line rather than compiled all at once, which makes testing and debugging faster.

High-Level Language: Python abstracts away low-level details like memory management, letting developers focus on logic.

Dynamically Typed: Variable types are determined at runtime, so there is no need to explicitly declare them.

Readable Syntax: Python uses indentation instead of curly braces to define code blocks, forcing clean, consistent formatting.

Extensive Standard Library: Often described as "batteries included," Python ships with built-in modules for everything from file handling to web requests.

Multi-Paradigm: Python supports procedural, object-oriented, and functional programming styles, giving developers flexibility.

Cross-Platform: Python code can run on Windows, macOS, and Linux with little to no modification.

Large Ecosystem: A vast collection of third-party libraries (NumPy, Pandas, TensorFlow, Django, Flask) makes Python suitable for nearly any domain.

Why This Matters for AI/ML

Python's combination of simple syntax and a mature ecosystem of scientific libraries (NumPy, Pandas, scikit-learn, TensorFlow, PyTorch) is why it became the dominant language for AI and Machine Learning. Researchers can express complex mathematical ideas in a few readable lines of code, while the heavy computation underneath is often handled by optimized C/C++ code wrapped inside these libraries — giving the best of both worlds: ease of writing and speed of execution.

5th concept: Compiler vs Interpreter

Every programming language needs a way to convert human-written code into instructions the machine can execute. There are two main approaches to do this: compiling and interpreting. The choice of approach affects speed, error handling, and how the language is typically used.

What is a Compiler?

A compiler takes the entire source code and translates it into machine code all at once, before the program runs. This produces a separate executable file (like a .exe). Once compiled, the program can be run directly by the machine without needing the original source code or the compiler again. If there is an error anywhere in the code, the compiler stops and reports it before execution begins — no part of the program runs until all errors are fixed.

Examples: C, C++, Java (compiles to bytecode), Rust, Go

What is an Interpreter?

An interpreter translates and executes code line by line, in real time, without producing a separate executable file. Each line is converted to machine instructions and run immediately, one after another. If an error exists further down in the code, the program will still execute all the correct lines before it reaches and stops at the faulty line.

Examples: Python, JavaScript, Ruby, PHP

Key Differences

Execution: Compiler translates the whole program at once; Interpreter translates and runs one line at a time.

Speed: Compiled programs run faster since translation happens only once, beforehand. Interpreted programs are typically slower since translation happens during execution, every time the program runs.

Error Detection: A compiler reports all errors at once, before execution. An interpreter stops at the first error it encounters during execution.

Output: A compiler produces a standalone executable file. An interpreter does not produce a separate file — it requires the interpreter program to run the code every time.

Portability: Compiled executables are often platform-specific (a Windows .exe will not run on Linux without recompilation). Interpreted code is generally more portable, as long as the interpreter is available on the target system.

Where Should a Compiler Be Used?

A compiled language is the better choice when:

Performance is critical: Applications like operating systems, game engines, and embedded systems need code to run as fast as possible, with direct hardware access.

The software will be distributed widely: A compiled executable can be shared without exposing the source code, and does not require the end-user to have any development tools installed.

Hardware-level control is required: Device drivers, microcontrollers, and real-time systems need precise, low-level control that compiled languages like C and C++ provide.

Example use cases: Windows/Linux operating systems, video games (GTA, PUBG), web browsers (Chrome's core engine), device drivers.

Where Is an Interpreter Preferred?

An interpreted language is the better choice when:

Rapid development and testing is needed: Code can be tested line by line without waiting for the entire program to compile, which speeds up experimentation.

Flexibility matters more than raw speed: Interpreted languages are easier to modify and debug on the fly, which is valuable in research and prototyping.

The task involves scripting or automation: Short scripts for file handling, web scraping, or task automation do not need the overhead of compilation.

The field is Data Science, AI, or Machine Learning: Quick experimentation, frequent code changes, and interactive environments (like Jupyter Notebooks) make interpreted languages like Python the natural choice.

Example use cases: Data analysis and ML model building (Python), dynamic websites (JavaScript running in the browser), automation scripts, educational/beginner programming.

A Simple Way to Remember

If the priority is speed and direct hardware control, use a compiled language. If the priority is flexibility, fast iteration, and ease of testing, use an interpreted language. This is also why Python — despite being interpreted and therefore slower on its own — remains the top choice for AI/ML: its core numerical libraries (NumPy, TensorFlow, PyTorch) are written in C/C++ internally, so Python provides easy syntax for development while the heavy computation still runs at compiled-language speed.